

Andrea Manzini

Green Computing

Misurare il

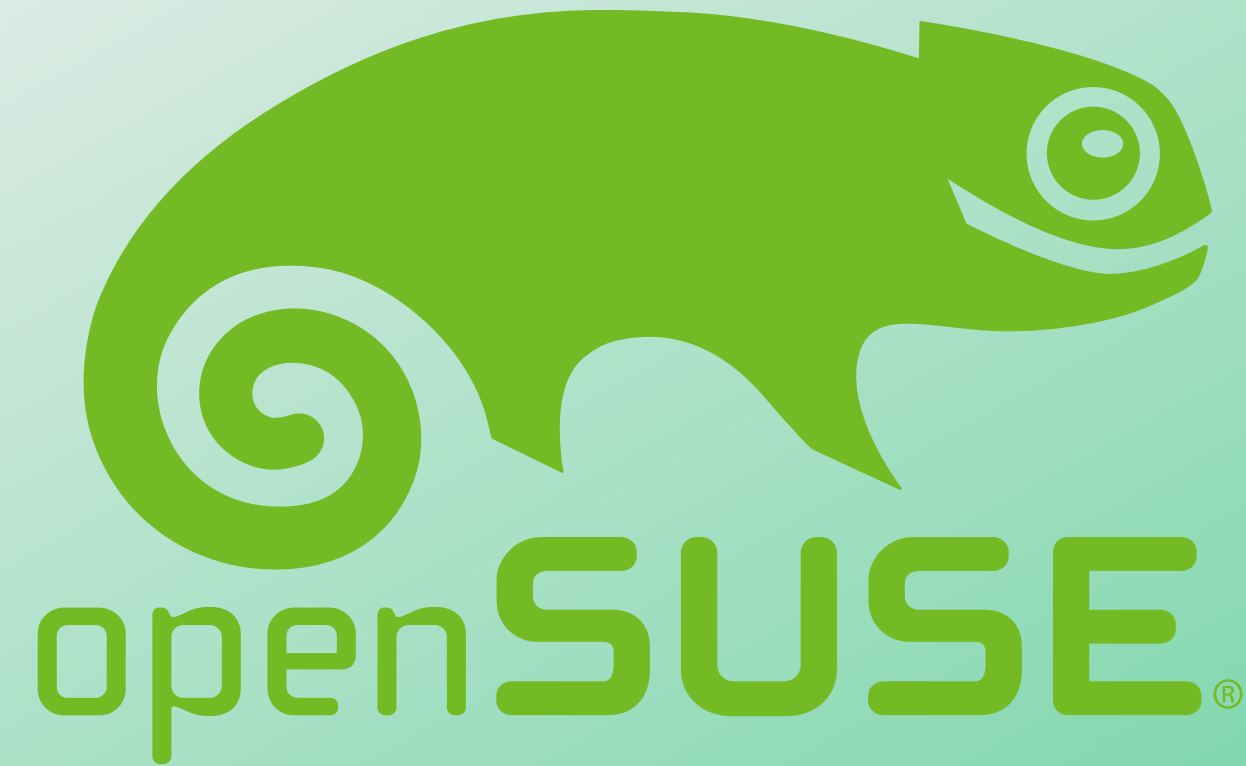
consumo

energetico

su Linux 🖱️ 🐧

Settembre 2026





Mi presento

Andrea Manzini

<https://ilmanzo.github.io>

- Veteran Unix Admin; BOFH
- Software engineer @SUSE
- Open Source Contributor 🙋
- Package Maintainer 📦

Avete uno smartphone, giusto? 📱

Immaginate: lascio il laptop acceso con un programma che tiene un core al 100%, dimenticato per un'ora, durante questa presentazione. Mi sapete dire grosso modo, tutta l'energia consumata dal laptop in quel tempo a quanta carica di un telefono medio equivale? 🔋

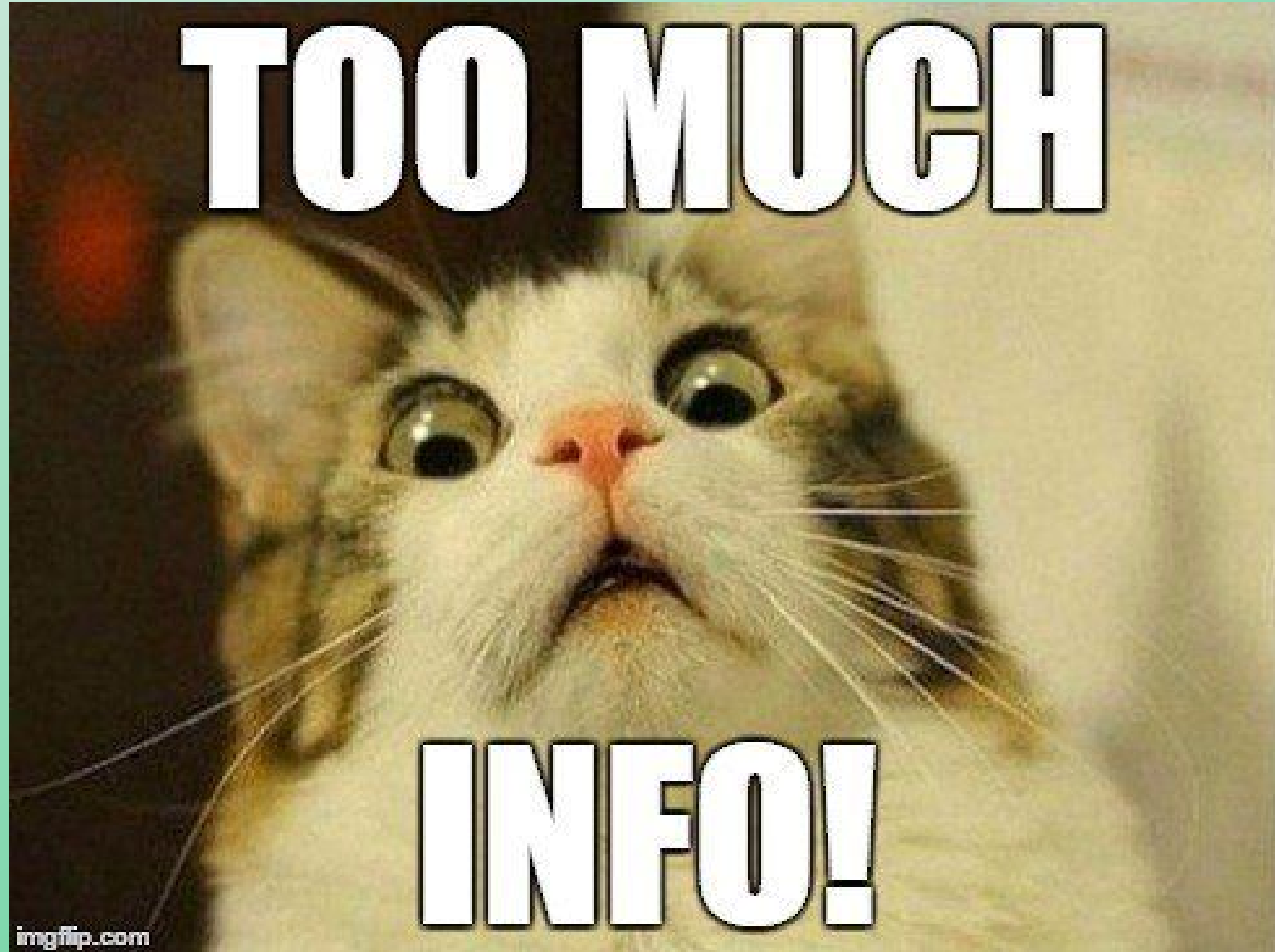
A. Meno del 20% (*giusto una sessione di social*)

B. Circa il 50% (*un pomeriggio intero di uso*)

C. Più del 90% (*quasi un'intera carica!*)

🙋 ...Votiamo ora!

🤔 ... Ma ve lo
dico alla fine!





Cosa vedremo oggi

- Perché è importante?
- Concetti Fondamentali: La differenza tra Potenza ed Energia.
- Come funziona? Uno sguardo alla tecnologia hardware/software
- Gli Strumenti del Mestiere
- Demo Live: Misuriamo un programma reale.
- Limiti e Conclusioni.

Perché l'efficienza energetica è importante?

Non è solo una questione di ecologia/moda "green"

➡️ Dispositivi Mobili & IoT

Meno consumo = più autonomia.

🖧 Data Center & Cloud

Piccoli risparmi su migliaia di server si traducono in milioni di euro risparmiati e un'impronta di CO₂ ridotta.

🚀 Prestazioni & Calore

Thermal throttling: la CPU rallenta per non surriscaldarsi, quindi un codice "caldo" è anche un codice più lento.

🤔 Watt, Joule?

Potenza [Watt]:

È la velocità con cui l'energia viene consumata. Es. la velocità della tua auto (sto andando a 90 km/h).

Energia [Joule]:

È la potenza moltiplicata per il tempo in cui viene erogata. Es. la distanza totale percorsa (ho percorso 200 km).

$$\text{Energia} = \text{Potenza} \times \text{Tempo}$$



Sì, ma in pratica?

ripasso di fisica: $1 \text{ W} = 1 \text{ J/s}$ (energia consumata al secondo)

$$1 \text{ kWh} = 1000 \text{ W} \times 3600 \text{ s} = 3,6 \cdot 10^6 \text{ J} = 3,6 \text{ MJ}$$

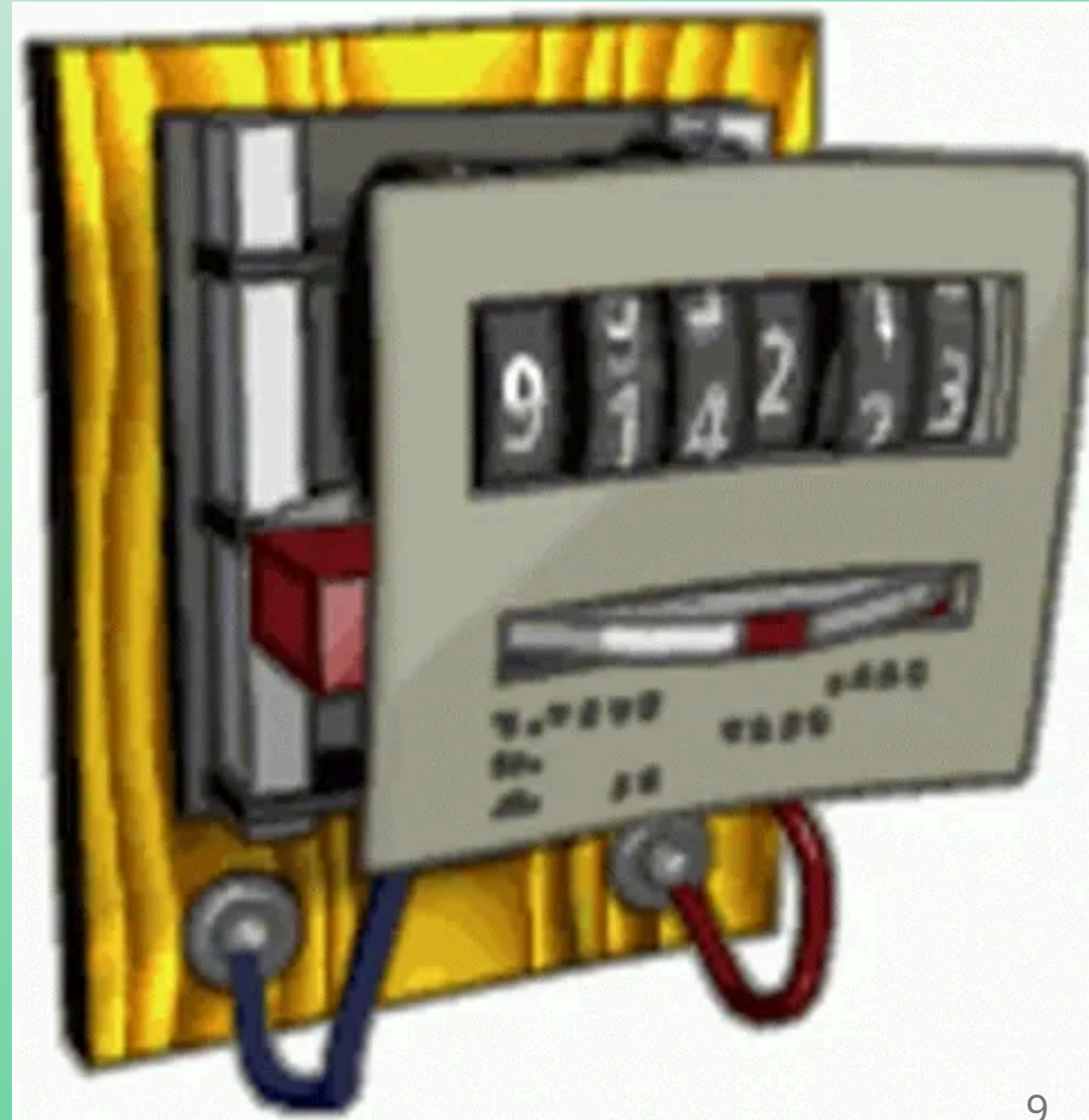
un kWh domestico in Italia costa circa 0,30 euro tasse incluse (ARERA, 2026)

- caricare un dispositivo a 30 W per 6 ore al giorno consuma 0,18 kWh al giorno, cioè $365 \times 0,18 = 65 \text{ kWh}$, circa **20 euro l'anno**
- un pc acceso + monitor consuma in media circa 100 W, se lo usiamo 8 ore al giorno spendiamo circa **120 euro l'anno**
- Un singolo datacenter può consumare anche 100 MW. A livello globale l'IEA stima i data center intorno ai **500 TWh** annui, in crescita verso ~950 TWh nel 2030.

La Magia sotto il cofano

Come può il software misurare l'hardware?

Il software da solo non può misurare l'assorbimento elettrico. Si affida a dei contatori e a modelli energetici esposti direttamente dal processore.



RAPL (Running Average Power Limit)

Cos'è? Un'interfaccia presente nelle CPU moderne (Intel e AMD) che fornisce stime sul consumo energetico.

È accurato? Non è un wattmetro, ma una stima prodotta dall'hardware stesso: dai contatori di attività e, dai [Skylake](#) in poi, dai regolatori di tensione integrati. Per i nostri scopi, è più che sufficiente.

Cosa misura? Espone diversi "domini" di consumo:

- PKG: L'intero socket della CPU.
- CORE: I core computazionali.
- UNCORE: La cache L3, i memory controller, ecc.
- DRAM: I moduli di RAM collegati.

RAPL in Linux: il framework powercap

Linux integra i dati di RAPL attraverso il framework powercap.

I dati sono accessibili direttamente tramite il filesystem virtuale, in:

```
/sys/class/powercap/
```

Dentro questa directory, troverete delle sottodirectory per ogni interfaccia (es. `intel-rapl:0`) con file come `energy_uj` (energia in **microjoule**) che possono essere letti.

Di seguito vediamo alcuni strumenti che semplificano la lettura e l'interpretazione di questi file.

Un primo script molto grezzo

```
import time

BASE = "/sys/class/powercap/intel-rapl/intel-rapl:0/"

def read(name):
    with open(BASE + name) as f:
        return int(f.read())

MAX = read("max_energy_range_uj")    # il contatore riparte da zero: % MAX

v0 = read("energy_uj")
while True:
    time.sleep(1)                    # ~1s fisso: J consumati in 1s = W
    v1 = read("energy_uj")
    print(f"Potenza media: {(v1 - v0) % MAX / 1e6:.1f} W")
    v0 = v1
```

powertop, powerstat, turbostat: i "cruscotti"

Esistono dei tool per monitorare il consumo energetico in tempo reale.

Ideali per:

- Identificare processi e driver che "svegliano" la CPU inutilmente.
- Avere una visione d'insieme del consumo del sistema.
- Applicare al volo ottimizzazioni a livello di sistema (scheda "Tunables").

```
sudo powertop          # diagnosi interattiva + Tunables
sudo powerstat -R 1    # W nel tempo, letti da RAPL
sudo turbostat         # W per package, frequenza, C-state
```

perf: Il Bisturi di Precisione

`perf` è lo strumento standard de facto per la profilazione delle performance (e dell'energia) su Linux.

Ideale per:

- Ottenere una misura precisa e ripetibile in Joule per l'esecuzione di un singolo comando.
- Confrontare il costo energetico di due versioni di un algoritmo.
- Integrare la misurazione energetica in script di test o CI/CD.

Perf in azione

Usare perf per misurare l'energia

Il comando `perf stat` esegue un programma e, al termine, riporta le statistiche raccolte, incluse quelle energetiche.

```
sudo perf stat -e [eventi] -- [comando]
```

Esempio pratico:

```
# Misura l'energia consumata da CPU (pkg) e RAM (dram)  
# durante l'esecuzione di 'mio_programma'  
  
sudo perf stat -e power/energy-pkg/ -- ./mio_programma
```

Dall'idea alla misura in 5 passi

- **Prepara l'ambiente:** chiudi le app inutili, fissa il governor
- **Misura a vuoto:** è system wide, l'idle va sottratto
- **Baseline:** misura la versione attuale, più volte
- **Ottimizza:** codice, flag di compilazione, algoritmo
- **Rimisura:** stesse condizioni, confronta i Joule

```
sudo cpupower frequency-set -g performance
sudo perf stat -e power/energy-pkg/ -- sleep 30      # baseline idle
sudo perf stat -r 5 -e power/energy-pkg/ -- ./prog    # 5 esecuzioni + deviazione standard
```

Demo: misuriamo un ordinamento

Creo un file di testo da 1 GB, in RAM

```
$ tr -dc 'A-Za-z0-9 ' < /dev/urandom | fold -w 80 | head -c 1G > /dev/shm/big.txt
$ wc -l /dev/shm/big.txt
13256071 /dev/shm/big.txt
```

```
$ time sort -S 4G -T /dev/shm --parallel=1 /dev/shm/big.txt > /dev/null
24.244 total
$ time sort -S 4G -T /dev/shm --parallel=16 /dev/shm/big.txt > /dev/null
6.597 total
```

La versione parallela finisce prima, e usa tutti i processori.

Consumerà di piú o di meno ?

Misura 1: sort single-thread

Comando:

```
sudo perf stat -r 5 -e power/energy-pkg/ -- \
  sort -S 4G -T /dev/shm --parallel=1 /dev/shm/big.txt > /dev/null
```

Performance counter stats for 'system wide' (media di 5 esecuzioni):

```
118.05 Joules power/energy-pkg/
6.91 seconds time elapsed
```

→ potenza media: $118,05 \text{ J} / 6,91 \text{ s} = 17,1 \text{ W}$

Misura 2: sort parallelo

Comando:

```
sudo perf stat -r 5 -e power/energy-pkg/ -- \
  sort -S 4G -T /dev/shm --parallel=16 /dev/shm/big.txt > /dev/null
```

Performance counter stats for 'system wide' (media di 5 esecuzioni):

```
58.01 Joules power/energy-pkg/
2.56 seconds time elapsed
```

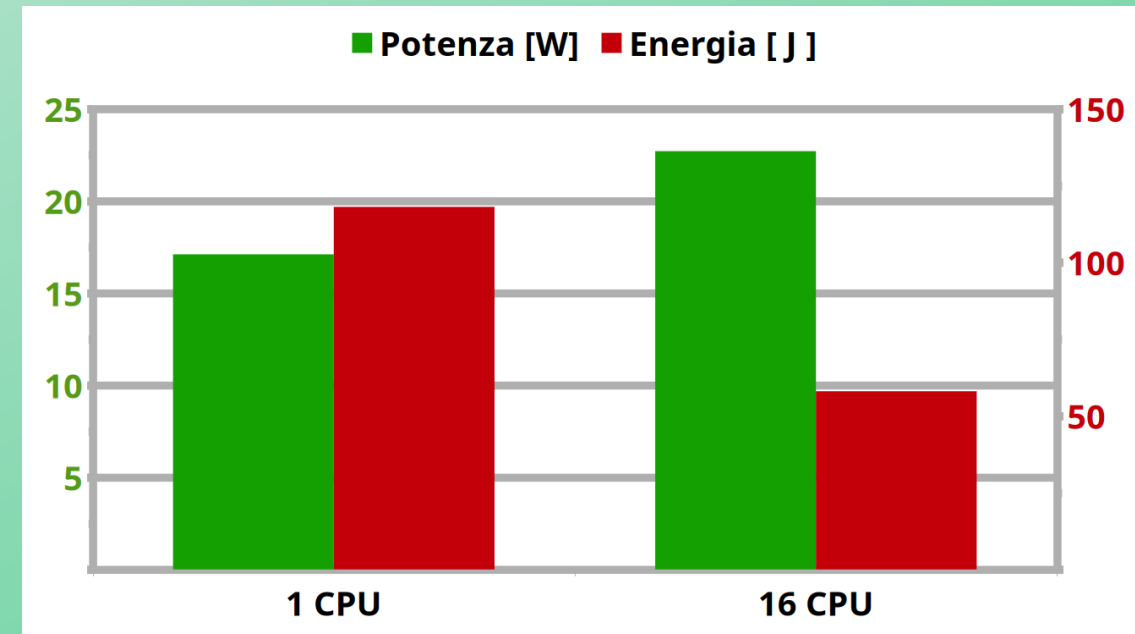
→ potenza media: $58,01 \text{ J} / 2,56 \text{ s} = 22,7 \text{ W}$ (più Watt del single-thread!)

Risultato

un risparmio energetico del ~51%!

	Energia	Tempo	Potenza
1 thread	118,05 J	6,91 s	17,1 W
16 thread	58,01 J	2,56 s	22,7 W

Più Watt, meno Joule: **race to idle**.



Il milionesimo numero primo



```
#!/usr/bin/env python3
from math import sqrt

def is_prime(n):
    for j in range(3, int(sqrt(n)) + 1, 2):
        if n % j == 0:
            return False
    return True

n, i = 1, 1    # il 2 è già contato
while n < 1_000_000:
    i += 2
    if is_prime(i):
        n += 1
print("Il milionesimo primo è", i)
```

```

fn is_prime(n: u64) -> bool {
    let sqrt_n = (n as f64).sqrt() as u64;
    for j in (3..=sqrt_n).step_by(2) {
        if n % j == 0 {
            return false;
        }
    }
    true
}

fn main() {
    let mut n = 1;    // il 2 è già contato
    let mut i = 1;

    while n < 1_000_000 {
        i += 2;
        if is_prime(i) {
            n += 1;
        }
    }
    println!("Il milionesimo primo è {}", i);
}

```



Pronti? Via! 1/2

```
$ sudo perf stat -e power/energy-pkg/ -- ./prime.py
```

Python

```
Il milionesimo primo è 15485863  
Performance counter stats for 'system wide':  
944.99 Joules power/energy-pkg/  
56.61 seconds time elapsed
```

Pronti? Via! 2/2

```
$ sudo perf stat -e power/energy-pkg/ -- prime_rs/target/release/prime
```

Rust

```
Il milionesimo primo è 15485863  
Performance counter stats for 'system wide':  
35.79 Joules power/energy-pkg/  
2.22 seconds time elapsed
```

Per questo task:

 impiega circa 25 volte più tempo e consuma circa 27 volte più energia di 

Ma attenzione al *perché*

	Energia	Tempo	Potenza
 Python	944,99 J	56,61 s	16,7 W
 Rust	35,79 J	2,22 s	16,1 W

La potenza è **quasi la stessa**: il linguaggio non cambia (di molto) quanto scalda la CPU, cambia soprattutto **per quanto tempo** la tiene occupata.

Disclaimer: CPython interpretato vs Rust in release. Il punto non è scoprire che "Python è lento", è mostrare che runtime e algoritmo si pagano in Joule.

Ok, ho misurato. E adesso cosa ottimizzo?

- **Race to idle:** finire presto consuma meno di rallentare
- **No alle attese attive:** `epoll` / `select` , non loop di polling
- **Raggruppa i wakeup:** timer coalescing (vedi `power top`)
- **Prima l'algoritmo:** un algoritmo migliore batte le micro-ottimizzazioni
- **Meno traffico DRAM:** cache-friendly = meno Joule (`power/energy-dram/`)

💡 Misura in **Joule per unità di lavoro:** l'unico numero confrontabile fra macchine diverse. Joule/Richiesta , Joule/Immagine , Joule/Gb

Cosa NON stiamo misurando

È importante essere consapevoli dei limiti di questo approccio.

RAPL è un modello, non la verità assoluta. L'accuratezza può variare leggermente tra le architetture.

Copertura parziale: RAPL misura CPU e RAM, ma non copre:

- GPU dedicata
- Dischi (SSD/HDD)
- Scheda di rete

Per un'analisi completa dell'intero sistema, sarebbero necessari strumenti esterni (wattmetri fisici).

E nel Cloud / Kubernetes?

Come facciamo a misurare l'energia in datacenter o nei pod di un cluster Kubernetes?

In ambienti Cloud Native isolare il consumo con `perf` è complesso. L'ecosistema sta adottando strumenti dedicati che esportano metriche energetiche (spesso per Prometheus):

- **Kepler** (Kubernetes-based Efficient Power Level Exporter): Un progetto Sandbox della CNCF che usa eBPF e i contatori RAPL per ripartire i consumi energetici (CPU, RAM, GPU) sui singoli Pod/Container.
- **Scaphandre**: Un agente di monitoraggio energetico open-source pensato per tracciare e **ripartire il consumo di energia dei server** tra i vari servizi in esecuzione, ottimo anche in ambienti containerizzati.

Sveliamo l'arcano

La risposta corretta era la **C: circa il 90%** 

- **I dati reali:** nella demo di prima, `prime.py` teneva un core impegnato al 100% per 57 secondi, niente altro: un carico identico a quello del quiz. Il laptop, in quel momento (idle compreso), consumava circa **16,7 W**.
- In un'ora: $16,7 \text{ W} \times 1 \text{ h} \approx \mathbf{16,7 \text{ Wh}}$.
- Una carica completa di uno smartphone medio vale circa $\mathbf{3,7 \text{ V} \times 5000 \text{ mAh} = 18,5 \text{ Wh}}$.
- **Quindi:** 16,7 Wh sono circa il **90%** di una carica. Basta un'ora di un programma dimenticato acceso per scaricare quasi un intero telefono.

E se considerassimo la "Scala Cloud"? ☁️

Cosa succede se questo banale bug o inefficienza finisce in produzione su un'infrastruttura di **10.000 istanze** attive 24/7/365?

- **Potenza IT sprecata:** $10.000 \times 16,7 \text{ W} = 167.000 \text{ W}$ (167 kW)
- **Alla presa,** con un PUE di 1,3 (raffreddamento incluso): $\sim 217,1 \text{ kW}$
- **Energia annua sprecata:** $217,1 \text{ kW} \times 8760 \text{ h} = 1.901.796 \text{ kWh}$
- **Costo annuo** (0,26 €/kWh, tariffa industriale, ARERA 2025): ben **494.000 €** buttati in bolletta! 💸

L'inefficienza del software sul singolo laptop sembra trascurabile, ma si paga a caro prezzo su larga scala.

Perché dobbiamo misurare?

- **L'hardware si vede:** se lasciamo l'acqua aperta o una lampadina accesa, ce ne accorgiamo.
 - **Il software è invisibile:** un algoritmo inefficiente o un loop infinito in un thread rimangono nascosti. **Senza dati, stiamo programmando al buio.**
- 💡 **Se non lo misuri, non puoi ottimizzarlo.** Strumenti come `RAPL` e `perf` servono proprio ad "accendere la luce".

Cosa portarsi a casa

- L'efficienza energetica è una metrica di qualità quanto le performance o la correttezza
- RAPL è la tecnologia chiave: un'interfaccia hardware esposta da Linux
- `powertop` : diagnosi generale, sprechi in background
- `powerstat` / `turbostat` : consumo del sistema in tempo reale
- `perf stat -r 5` : misure precise e ripetibili, **sottraendo sempre l'idle**
- *Finire in fretta* è spesso la strategia più efficiente 🏎️

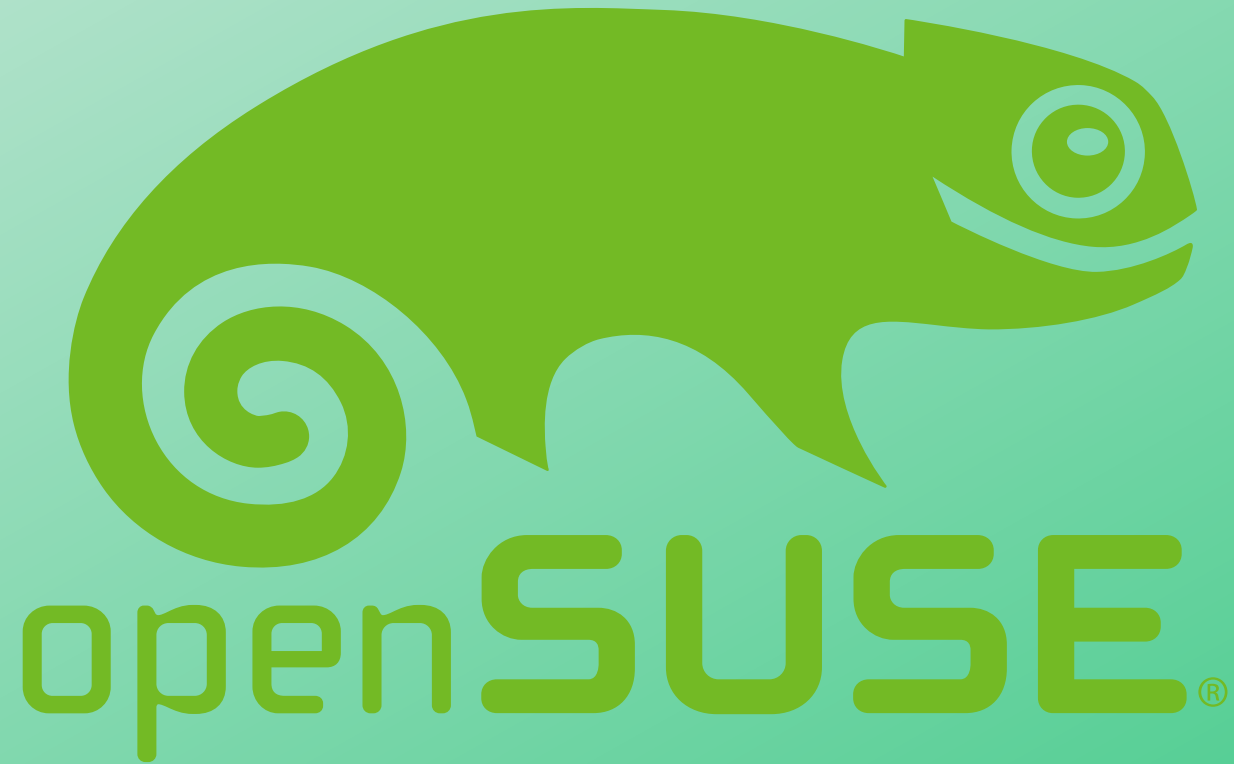
Grazie per l'attenzione!

Andrea Manzini

<https://ilmanzo.github.io>

Domande e Risposte

- *"ma se uso lo schermo nero, risparmi?"* → OLED sì, LCD no
- *"funziona anche su ARM?"*
- *"E per le GPU?"*



Risorse

- Wiki di `perf`: <https://perfwiki.github.io/>
- Pagine man: `powertop`, `powerstat`, `turbostat`, `perf`
- RAPL: [come funziona](#) (Intel)
- Pereira et al., *Energy Efficiency across Programming Languages* (SLE 2017): i dati dietro 🐍 vs 🦀
- [Green Software Foundation](#): pratiche, corsi, Software Carbon Intensity
- [Brendan Gregg](#): performance su Linux
- `s-tui`: TUI per stressare e monitorare la CPU (frequenza, temperatura, consumo)